

Laghlali Noah
Rochedreux Hugo
Klingler Emma

RA-IL2

THE SILENT HUNT



2025-2026

Tuteur : Monsieur Olivier Perrin

Sommaire

Sommaire	2
1. Introduction	4
2. Analyse du projet	5
2.1 Évolution par itérations	5
3. Réalisation	6
3.1 Architecture logicielle	6
3.1.1 Description	6
3.1.2 Pourquoi cette structure a été choisie	8
3.2 Arbre de comportement (BT)	9
3.2.1 Sélecteur de poids	10
3.2.1.1 Sélection du noeud	11
3.2.1.2 Exécution du noeud	11
3.2.2 Utility System	12
3.2.2.1 Appliquer l'Utility au chasseur	12
3.2.2.2 Formaliser le chasseur	14
4. Tests de validation	16
5. Difficultés rencontrées	17
6. Itération de ce semestre	18
Itération 1	18
Itération 2	21
Itération 3	24
Objectifs	24
Avant Itération	24
Déroulement	25
7. Répartition du travail	28
8. Planning de déroulement du projet durant les 3 itérations de ce semestre	29
9. Élément original	30
Noah Laghlali	30
Emma Klingler	31
Hugo Rochedreux	32
10. Objectifs à atteindre pour la fin du projet	33
10.1 IA du chasseur (priorité n°1)	33
10.2 IA du lapin	33
10.3 HUD et visuels	33
10.4 Missions secondaires	33
10.5 Multijoueur	34
10.6 Carte finale	34
10.7 Finition et optimisation	34

11. Planning pour les trois itérations restantes	35
11.1 Itération 4 : Ajustement des poids	35
11.2 Itération 5 : IA adaptative et apprentissage léger	35
11.3 Itération 6 : Amélioration final et préparation de la soutenance	36
Conclusion	37

1. Introduction

Ce rapport présente l'état de notre projet tutoré The Silent Hunt en fin de semestre. Ce projet a pour but de suivre les objectifs, la mécanique du jeu ainsi que les choix techniques définis par l'étude préalable effectuée en début de semestre.

The Silent Hunt est un jeu de survie développé sur la plateforme de jeu Roblox à l'aide du langage Lua. Le joueur incarne un lapin évoluant dans une forêt, poursuivi par un chasseur contrôlé par une intelligence artificielle. Le but du jeu est de survivre le plus longtemps possible en gérant plusieurs ressources essentielles comme la vie, la faim et le stress, tout en utilisant l'environnement pour se cacher et fuir.

L'un des éléments principaux du projet est l'intelligence artificielle du chasseur. Celle-ci repose sur un Behaviour Tree, utilisé comme structure de décision principale. Celui-ci est complété par une approche de type Utility AI, permettant d'ajuster dynamiquement les priorités des actions selon le contexte de jeu, et ainsi d'adapter le comportement du chasseur au joueur.

Ce document présente l'analyse du projet, le déroulement des différentes itérations du semestre, les choix techniques réalisés, la répartition du travail entre les membres du groupe ainsi que les objectifs prévus pour la suite du développement.

2. Analyse du projet

Le lapin constitue le cœur du gameplay :

- déplacements (marcher, courir, sauter)
- actions principales : manger, se cacher
- caméra en première personne pour l'immersion
- gestion de la faim, de la santé, et de la mort
- HUD synchronisé entre client et serveur

Le chasseur, de son côté, utilise un Behaviour Tree lui permettant :

- de patrouiller, enquêter et poursuivre
- de poser des pièges, recharger, chercher des munitions
- d'adopter un comportement crédible et réactif

L'environnement inclut une carte de test avec une petite forêt, des zones, et des terriers sont prévus, servant à la fois de cachettes et de téléportation.

Le jeu intègre un mode multijoueur, avec synchronisation client/serveur, des objets interactifs (carottes) et un lobby permettant de créer ou rejoindre une partie, gérer les joueurs et configurer le nombre de lapins IA. Une IA de lapin simple a également été ajoutée pour plus tard entraîner le chasseur.

2.1 Évolution par itérations

Itération 1 : Prototype solo

Les objectifs initiaux portaient sur le déplacement, la survie, et une IA simple du chasseur (FSM).

Itération 2 : Interaction et premières mécaniques avancées

L'itération devait introduire le système multijoueur, la détection de bruit et surtout le début de BT pour le chasseur.

Itération 3 : IA avancée (partiellement atteinte)

L'objectif était d'ajouter le système de piège, de terrier et d'améliorer le BT du chasseur.

3. Réalisation

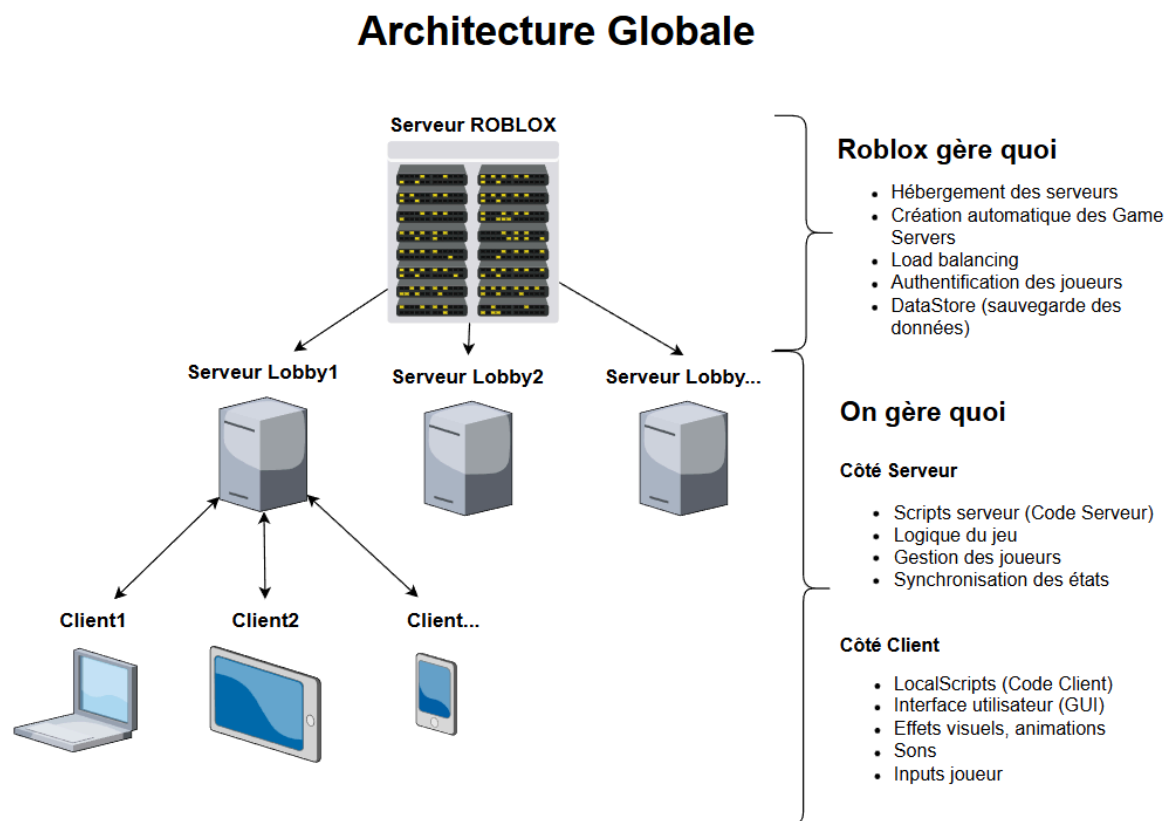
3.1 Architecture logicielle

3.1.1 Description

L'architecture de The Silent Hunt repose sur trois outils principaux : **Roblox Studio**, **Rojo** (VS Code) et **GitHub**.

Roblox Studio est un environnement de développement (IDE) spécialisé dans la conception de jeux vidéo.

Voici l'architecture de roblox pour mieux comprendre.



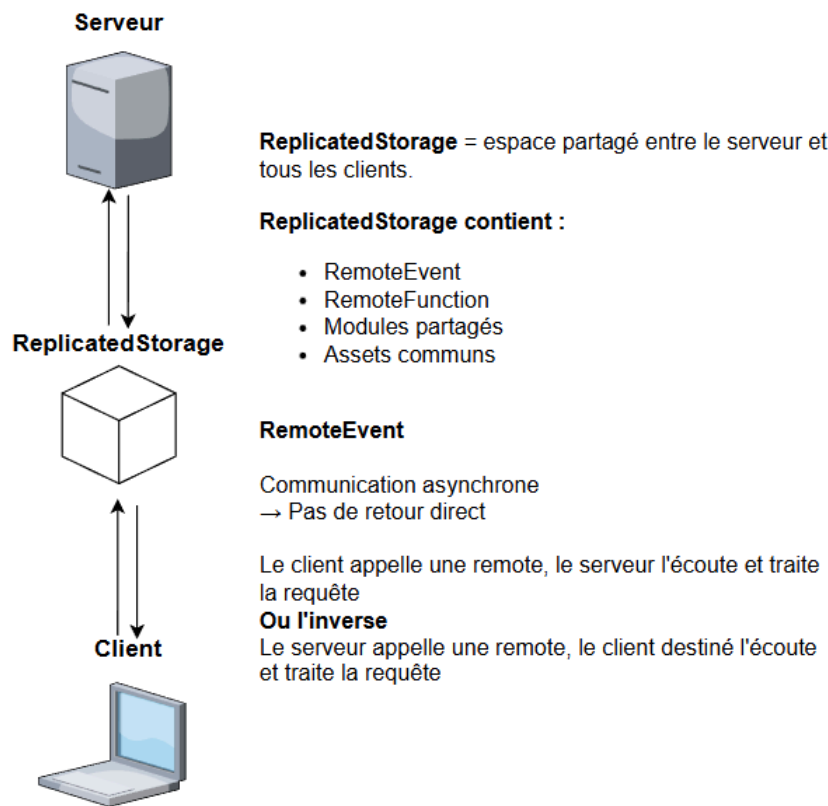
1. Architecture de Roblox

Rojo synchronise automatiquement vers Roblox Studio, ce qui évite les scripts dispersés et difficiles à retrouver dans Roblox Studio.

GitHub sert au versioning, au travail en groupe et à la sauvegarde des différentes itérations du projet.

Une partie importante de l'architecture est la communication entre serveur et clients. Pour mieux comprendre, voici un schéma des échanges :

Communication



2. Système de communication entre client et serveur

En résumé, **ReplicatedStorage** est un espace partagé. Il est composé de **RemoteEvents**, ce qui permet la communication entre serveur et clients.

Dans Roblox, le serveur est l'instance principale : IA, logique du jeu, état global, etc.

Donc si le serveur s'arrête :

- L'IA du chasseur s'arrête aussi (scripts serveur stoppés).
- Les joueurs sont automatiquement déconnectés (Roblox ferme la session).
- L'état du jeu est perdu.
- Aucun script client ne peut continuer la partie, car ils dépendent des RemoteEvents/RemoteFunctions du serveur.

3.1.2 Pourquoi cette structure a été choisie

Séparation claire des responsabilités :

- Serveur : IA, règles du jeu, systèmes critiques.
- Client : interface, contrôles, caméra.
- ReplicatedStorage : modules partagés (RemoteEvent).

Cette organisation suit les standards Roblox, donc moins de conflits, meilleure lisibilité, comportement multijoueur cohérent.

Rojo + VS Code

- Édition du code propre, structurée en dossiers.
- Synchronisation automatique, donc aussi rapide que d'éditer dans Roblox Studio.

GitHub

- Versioning, historique clair.
- Travail en équipe facile.

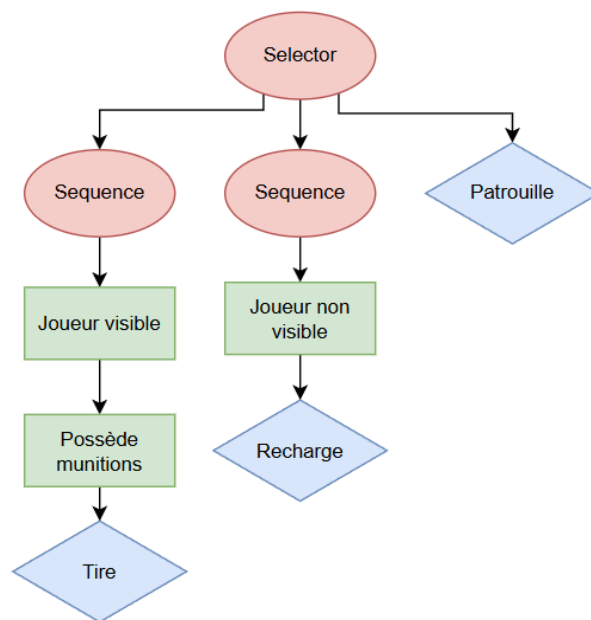
Nous avons décidé de choisir Roblox Studio car c'est un logiciel assez simple à prendre en main comparé à des logiciels comme Unity ou Unreal Engine, plus puissant mais plus compliqué.

3.2 Arbre de comportement (BT)

Un arbre de comportement ou Behaviour Tree (BT), c'est une structure modulaire, un arbre qui fonctionne avec des nœuds de type **sélection**, **action** ou **condition**.

Il s'exécute à chaque frame et va exécuter un nœud de type sélection, qui va choisir un nœud à exécuter, et ainsi de suite jusqu'à exécuter un nœud action qui retourne soit **SUCCESS**, **FAILURE** ou **RUNNING**.

Si on prend un exemple simple pour le chasseur :



3. BT simple du chasseur

Donc à chaque exécution, notre arbre va être déroulé depuis la racine. Notre arbre va être parcouru de gauche à droite jusqu'à ce qu'un nœud soit valide. Si on prend notre *Image* ci-dessus, notre arbre va regarder si le joueur est visible, si le chasseur possède des munitions et si oui, alors il va tirer. Sinon il passe à la branche suivante jusqu'à ce qu'une branche renvoie **RUNNING** ou **SUCCESS**.

On a donc une structure très modulaire et assez simple, on pourrait dire parfaite pour notre chasseur.

Dans un arbre de comportement les priorités sont statiques. Elles sont directement inscrites dans notre arbre. Cela le rend très lisible mais en pratique c'est très limitant.

Or dans notre jeu, une action peut avoir une priorité différente en fonction du contexte. Cela revient donc souvent à dupliquer une section de notre arbre en changeant les conditions, ce n'est pas pratique et ne résout pas le problème.

En réalité, une décision n'est jamais binaire, si on prend notre chasseur en exemple (*Image 3. BT simple du chasseur*), l'action Recharge ne se fait que lorsque le joueur n'est pas visible. Or si le chasseur ne possède plus de balle dans son chargeur mais voit un joueur, il ne pourra jamais recharger ce qui pose un problème.

La solution à tous ces problèmes est l'intégration de l'utility.

(Plus de détails 10.2, 10.3 [Documentation](#))

3.2.1 Sélecteur de poids

L'objectif d'un sélecteur de poids (Weighted Selector) est de sélectionner une action parmi ses nœuds enfants en fonction de leur poids.

Un sélecteur de poids possède une liste d'enfants composée de:

- Un nœud Action cf(ActionChasseur)
- Une variable **bloquer** qui permet de bloquer l'action si jamais elle est en cours. Par exemple : une fois que le nœud Recharger est en cours, le sélecteur ne changera pas de nœud jusqu'à la fin de l'action.
- Une variable **priorité** qui permet de gérer la priorité de chaque action. Par exemple, si le nœud **Poursuivre** est de priorité 2, le nœud Explorer de priorité 1 et qu'il explore, il peut être interrompu par la poursuite, mais inversement non.
- Une variable **poids** qui est l'utility de l'action

Pour résumer notre Behaviour tree boucle à chaque frame du jeu, et va calculer les poids de chaque action ensuite il va exécuter le sélecteur de poids qui va donc sélectionner l'action à choisir.

3.2.1.1 Sélection du noeud

Le sélecteur va regarder quel nœud enfant a le **poids** le plus élevé ainsi que l'ensemble des **poids** de tous ses enfants.

Par exemple, **60%** du temps le nœud avec le poids le plus fort sera choisi, sinon **40%** du temps un enfant aléatoire en fonction des poids sera choisi, cela permet au chasseur d'être 60% du temps "parfait" et 40% plus humain en choisissant de manière plus aléatoire.

3.2.1.2 Exécution du noeud

Premièrement, le sélecteur vérifie si la variable **bloquer** du nœud en cours est à vrai, si oui, il continue d'exécuter le nœud, sinon, il va sélectionner un nouveau nœud et va comparer sa priorité avec celle du nœud courant. Si le nouveau est plus prioritaire alors il va le remplacer sinon, le nœud courant n'est pas remplacé.

Un système d'hystérésis a été mis en place afin d'éviter des changements d'action trop fréquents. Le sélecteur ne change de comportement que si la nouvelle action possède un poids suffisant par rapport à l'action actuelle (seuil fixé à 85 %). Cela permet d'éviter des alternances constantes entre deux actions de même importance.

Enfin, afin d'augmenter les performances et de réduire les calculs, nous avons ajouté un temps pour la réévaluation du nœud. Le sélecteur change une fois toutes les 0.25 secondes, soit 4 fois par seconde. En dessous de 4 fois par seconde, cela serait inutile car invisible à l'œil nu.

3.2.2 Utility System

Un utility system est une méthode de prise de décision utilisée en IA dans les jeux pour mesurer à quel point une action est pertinente par rapport à un moment donné.

Au lieu de décider si une action est valide ou non comme vu dans la section de l'arbre de comportement, on attribue un score (une utilité) à chaque action possible. On compare ensuite ces scores et on sélectionne l'action correspondante grâce à notre **Weighted Selector**.

L'idée, comme vu précédemment, est qu'il n'y a presque jamais une seule "bonne" décision mais il existe souvent plusieurs options raisonnables.

L'utility system permet donc de mesurer la fiabilité d'une action et de choisir la meilleure.

3.2.2.1 Appliquer l'Utility au chasseur

Pour chaque action possible, on calcule une valeur d'utilité basée sur différents facteurs.

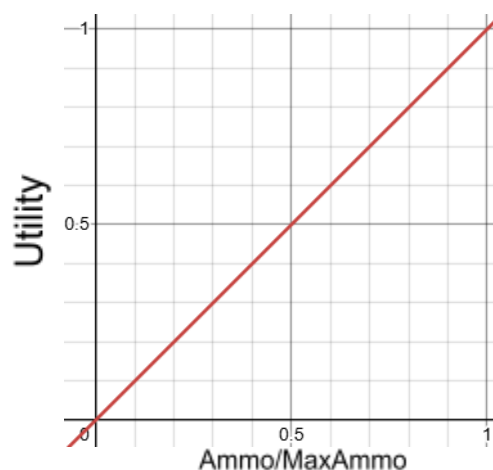
Prenons comme exemple le rechargement (Reload).

On peut premièrement mesurer certains attributs du chasseur :

- **Ammo** : nombre de balles dans le fusil du chasseur
- **MaxAmmo** : nombre de balles maximum dans le fusil du chasseur

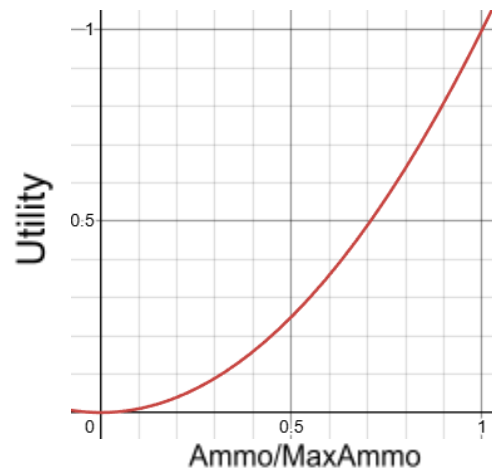
On pourrait alors créer une formule simple :

$$Utility(Recharge) = 1 - (Ammo/MaxAmmo)$$



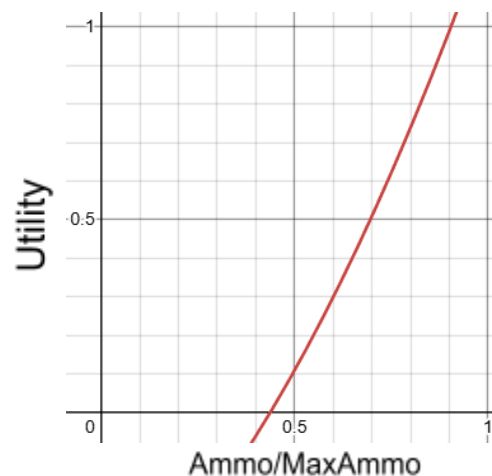
Avec notre formule simple on peut voir un problème. Si notre chasseur possède encore beaucoup de balles dans son chargeur, il y a quand même une chance qu'il recharge. C'est pourquoi on peut modifier l'allure de la courbe en appliquant la fonction puissance, ce qui nous donne une nouvelle formule.

$$Utility(Recharge) = 1 - (pow(Ammo/MaxAmmo))$$



Cette nouvelle courbe est plus intéressante car l'utilité augmente de manière plus douce au début et plus forte à la fin. On peut bien sûr aller encore plus loin en ajoutant quelques ajustements à notre fonction.

$$Utility(Recharge) = 1 - (pow(Ammo/MaxAmmo)+0.4)-0.7$$



Maintenant cette courbe est encore plus intéressante. Lorsque le chargeur du chasseur est rempli l'utilité est à 0 et c'est seulement quand le chargeur est à moitié vide que l'utilité va augmenter de manière exponentielle.

Bien évidemment ce n'est qu'un exemple simple, en effet on pourrait encore améliorer cette fonction. Une façon de faire serait d'ajouter de nouveaux facteurs comme la fatigue du chasseur ou alors la distance avec un joueur cible.

3.2.2.2 Formaliser le chasseur

L'utilité system nous oblige donc à formaliser notre chasseur. La première étape est d'énumérer les différentes actions du chasseur.

- Attaquer
- Tirer
- Recharger
- Ravitailler
- Poursuivre Cible
- Placer Piège
- Patrouiller

Ensuite, il faut énumérer les attributs du chasseur qui seront nos fameux facteurs pour le calcul d'utilité.

- | | |
|--------------------|---|
| - Fatigue | F |
| - Patience | P |
| - Agressivité | A |
| - BallesChargeur | B |
| - BallesInventaire | I |
| - NombrePièges | T |
| - VieLapin | V |
| - DistanceLapin | D |
| - LigneDeVue | L |
| - Position | O |

Les lettres représentent les curseurs dans [Desmos](https://www.desmos.com/calculator) (site web pour représenter le calcul de poids).

Problème : comment comparer les balles du chargeur avec la fatigue du chasseur ou d'autres attributs ?

On ne peut pas comparer des unités différentes directement.

Solution : **normaliser toutes les valeurs entre 0 et 1.**

Par exemple :

- 0 = pas du tout intéressant
- 1 = extrêmement intéressant

Si on prend en exemple **BallesChargeur**, l'attribut représente le nombre de balles dans le chargeur du chasseur. Avec la normalisation, on aurait notre fameux *Ammo/MaxAmmo*, une valeur comprise entre 0 et 1.

(Détails [Documentation](#))

4. Tests de validation

Les tests ont été menés selon deux approches complémentaires :

Des tests unitaires simples ont été réalisés dans VS Code ainsi que des tests en conditions réelles dans Roblox Studio.

Les tests en jeu ont permis de valider les mécaniques principales :

- déplacements du lapin et gestion correcte des collisions
- déclenchement de la mort lorsque la vie atteint 0
- diminution progressive de la faim et restauration avec les carottes
- affichage cohérent des jauges dans le HUD

Tests de l'IA du chasseur :

- transitions correctes entre patrouille, investigation et poursuite
- cohérence de la vision et perte de la cible après un délai
- vérification de la pose de pièges

Tests multijoueur :

- synchronisation des jauges (vie, faim)
- fonctionnement des objets interactifs
- lobby fonctionnel (rejoindre, exclure, lancer une partie)
- comportement du chasseur face à plusieurs cibles

Tests de performance :

- les scripts ne provoquent pas de ralentissements
- l'IA reste stable avec plusieurs joueurs
- le serveur ne rencontre pas d'erreurs majeures

Ces tests ont été répétés à chaque itération afin de valider les nouvelles fonctionnalités et garantir la stabilité du projet.

5. Difficultés rencontrées

Prise en main du code

- Apprentissage du langage Lua
- Compréhension du modèle client/serveur Roblox
- Découverte des modules, RemoteEvents et de Rojo

Organisation de l'équipe

- Absences fréquentes
- Équipe au complet seulement la dernière semaine
- Nécessité de revoir les priorités et de réduire certaines fonctionnalités

Complexité technique de l'IA du chasseur

- Mise en place d'une IA et d'Utility (ajustement des poids)
- Équilibrage du Behaviour Tree pour éviter une IA trop faible, incohérente ou imbattable
- Nombreux tests nécessaires pour stabiliser le comportement

Ce que nous avons appris

- Travailler efficacement avec un nouveau langage
- S'adapter aux imprévus d'équipe et réorganiser un projet en conséquence
- Concevoir une IA avancée
- Tester, ajuster et itérer rapidement pour obtenir un comportement cohérent
- Prioriser les fonctionnalités essentielles quand le temps est limité

6. Itération de ce semestre

Itération 1 :

Objectif de l'itération

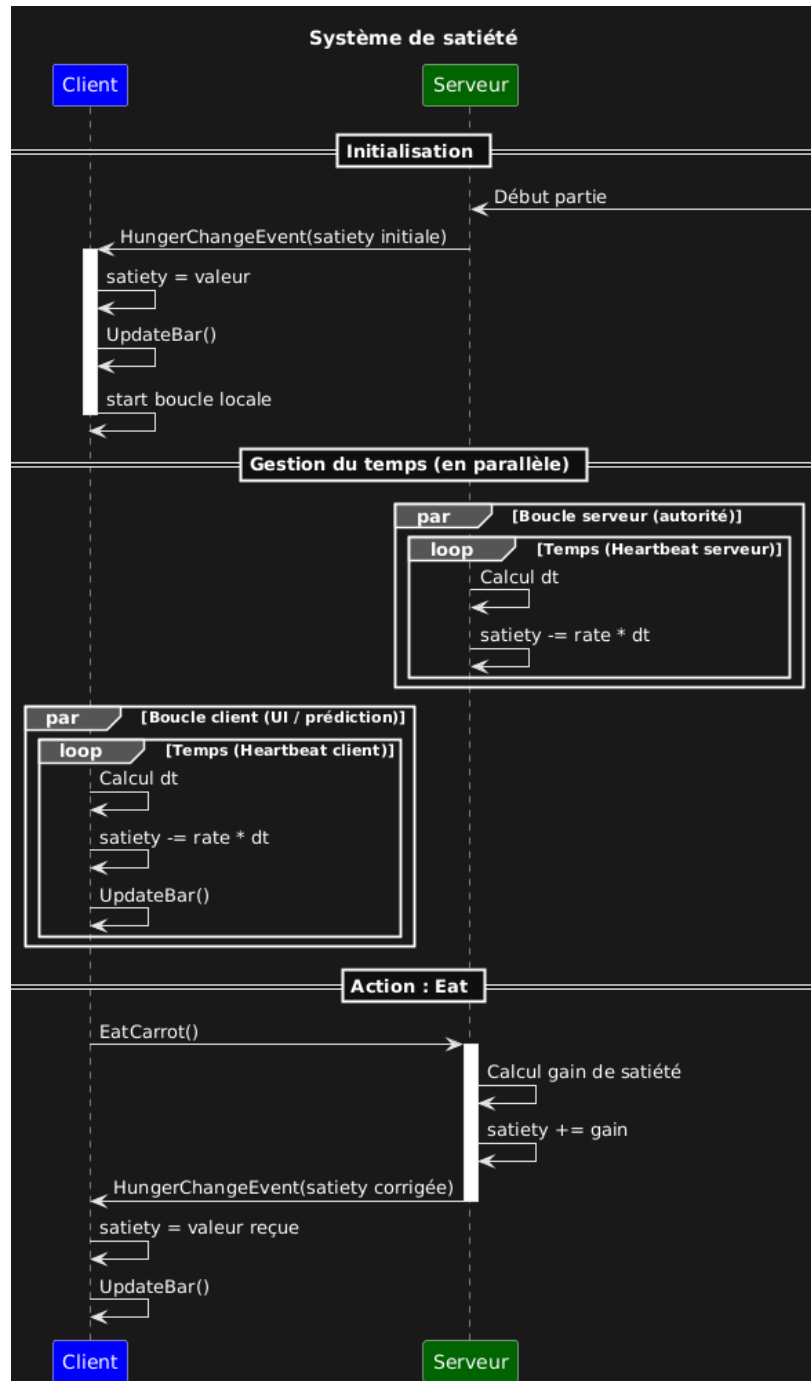
L'objectif était de produire une première version jouable de The Silent Hunt, en posant les bases du gameplay, de l'architecture et des systèmes essentiels.

Prototype fonctionnel

Nous avons suivi le plan établi lors de l'étude préalable et obtenu un prototype intégrant le lapin, le chasseur, la gestion de la vie et de la faim, ainsi qu'une première carte de test.

Système du joueur (le lapin)

- Implémentation d'un lapin jouable et animé
- Déplacements dans l'environnement
- Deux jauges : vie et faim
- La faim diminue progressivement, et la carotte permet de la restaurer
- HUD opérationnel grâce à un système de réplication serveur/client



4. Diagramme de séquence du système de satiété

Système du chasseur

- Introduction du chasseur avec un Behaviour Tree simple
- Patrouille aléatoire

- Détection du lapin et déplacement vers lui
- Attaque de base (un message console)

Carte de test et validation du gameplay

Une petite carte a permis de valider les mécaniques principales.

Un bug notable a été identifié : le lapin peut parfois “voler” dans les airs, un problème prévu pour la prochaine itération.

Compétences

Cette phase a permis à Noah et Emma de se familiariser avec Lua et l’environnement Roblox, en comprenant mieux les interactions entre scripts serveur, scripts client et classes personnalisées.

Itération 2

Objectifs de l'itération

L'objectif principal de l'itération II était de développer une intelligence artificielle du chasseur plus aboutie et jouable. Cette phase avait pour objectif de permettre de passer d'un prototype simple à une version plus proche de la vision définie dans l'étude préalable, avec un chasseur capable de détecter, poursuivre et attaquer le joueur de manière cohérente.

État de la version

À la fin de cette itération, l'IA du chasseur est fonctionnelle et intégrée au gameplay. Le chasseur est désormais capable de :

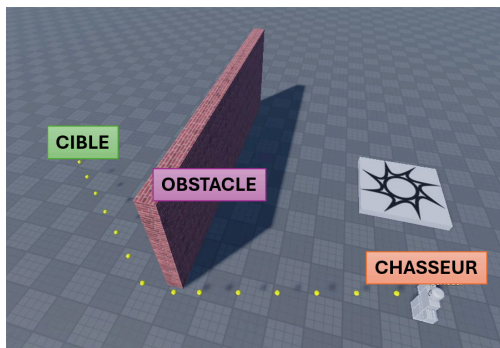
- détecter le lapin grâce à un système de vision basé sur un champ de vision et du raycasting ;
- mémoriser la dernière position connue du lapin ;
- poursuivre le joueur en utilisant le PathfindingService ;
- attaquer au corps à corps et à distance ;
- gérer un chargeur et une réserve de munitions ;
- se recharger et se ravitailler ;
- poser un premier type de piège basé sur la mémoire.

Pathfinding :

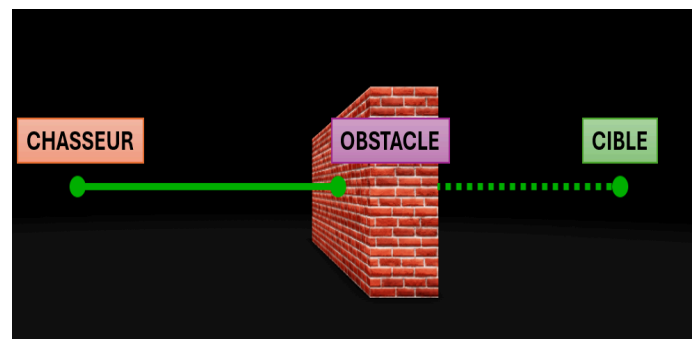
Le pathfinding permet au chasseur de calculer automatiquement un chemin vers le joueur en contournant les obstacles. Ainsi, il ne se déplace pas en ligne droite mais suit un trajet réaliste adapté à l'environnement.

Raycasting :

Le raycasting permet de vérifier la ligne de vue entre le chasseur et le lapin. Si un obstacle bloque le rayon, le tir est empêché, ce qui évite que l'IA tire à travers les murs.

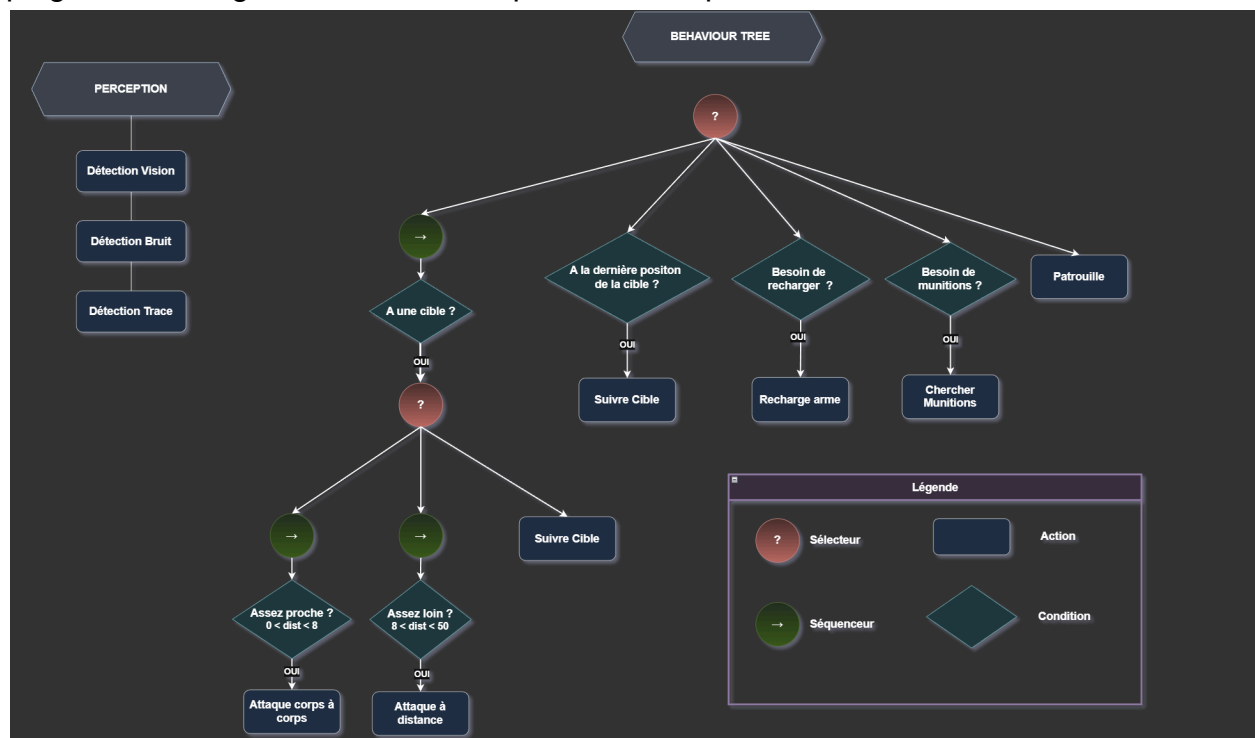


5. Fonctionnement du pathfinding



6. Fonctionnement du raycasting

Le Behaviour Tree a été amélioré afin de mieux gérer les priorités entre les différentes actions du chasseur, comme le combat, la poursuite, le rechargement ou la pose de pièges. Cette organisation rend l'IA plus lisible et plus facile à faire évoluer.



7. Schéma du BT sans poids

Le système multijoueur ainsi que la synchronisation client/serveur sont également opérationnels, ce qui permet un comportement cohérent du chasseur pour tous les joueurs connectés.

Fonctionnalités terminées	Fonctionnalités non terminées
Behaviour Tree amélioré	Détection du bruit
Détection du lapin par vision	Détection des traces
Mémorisation de la cible	Animations avancées du chasseur
Poursuite par pathfinding	Tests unitaires complets
Attaque à distance avec gestion du cooldown	Systèmes de buissons (cachettes)
Gestion des munitions et du chargeur	Intégration complète d'une Utility AI
Rechargement et ravitaillement	
Début du système de pièges	
Multijoueur et synchronisation	

Lien avec l'étude préalable

Les développements réalisés durant l'itération sont cohérents avec l'étude préalable. L'IA repose bien sur un Behaviour Tree. Même si certaines fonctionnalités restent à développer, la structure actuelle correspond à la vision initiale du projet.

Itération 3

Objectifs

Les objectifs de cette itération sont :

- Des pièges fonctionnels

Le chasseur doit pouvoir placer un nombre limité de pièges. Si un lapin tombe sur un piège, le lapin perd de la vie et le chasseur récupère un piège.

- Un terrier

Un gros terrier relié par des entrées partout sur la carte. Un lapin peut entrer dans le terrier, dedans sa faim augmente, il peut aussi sortir où il veut.

- Le calcul dynamique des poids

Avoir une IA du chasseur plus humaine et plus réaliste avec des poids sur chaque action.

Avant l'Itération

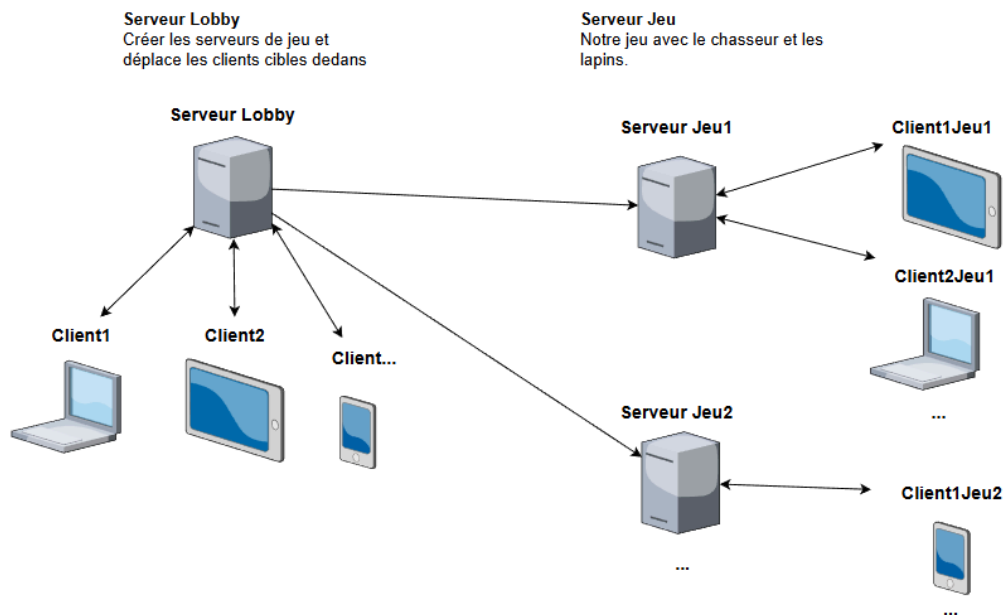
(Hugo Rochedreux)

A la fin de l'itération 2, nous avons remarqué que notre projet est un peu en retard par rapport aux autres, en grande partie due à l'absence de Noah pour raison médicale. C'est pourquoi, entre l'itération 2 et 3, j'ai décidé de rajouter des éléments plus graphiques qui prennent du temps.

J'ai donc ajouté :

- Modèle 3d chasseur
- Animations chasseur
- Lumière et ambiance
- Système de lobby

Les trois premiers points sont purement graphiques je ne vais pas les détailler, mais le système de lobby si.



8. Schéma de fonctionnement du lobby

Pour résumer l'image ci-dessus, quand un joueur se connecte au jeu, il est déplacé dans le serveur Lobby. Dedans, il peut créer un groupe, les autres peuvent le rejoindre et il peut commencer la partie. Quand la partie est lancée, le serveur Lobby va créer un serveur de jeu et déplacer tous les clients du groupe dedans.

Dans notre serveur de jeu, on va récupérer les informations telles que le nombre de joueurs, et on va attendre que tous les joueurs se connectent bien avant de lancer.

Bien sûr, si jamais un joueur met plus de 15 secondes à se connecter, la partie va commencer sans lui. Et si après les 15 secondes il se connecte il sera redirigé vers le serveur Lobby car la partie est en cours.

Déroulement

L'objectif principal de l'itération 3 est le calcul dynamique des poids (*cf Utility System*).

Pour ce faire nous avons commencé par énumérer chaque action et attribut de notre chasseur (*cf Formaliser le chasseur*) et ensuite réfléchi à chaque utility.

Nous avons donc créé les formules d'utility suivantes :

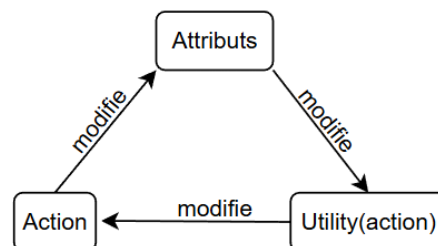
- Attaque = $A * (1-D) * (1-F)$
- Tir = $A * L * (B) * \max(0, 1 - \text{abs}(D-0.3)/0.6)$
- Placer Piège = $P * (1-F) * (1-D) * T$

- Recharge = $(1-B)*I$
- Ravitaillement = $(1-I)*(1-L)$
- Poursuite cible = $(1-F)*P*L$
- Poursuite Position = $P*(1-L)*(1-F)/2*O$
- Patrouille = $(1-L)*(1-\max(\text{Recharge}, \text{Ravitaillement}, \text{Poursuite Position}))$

Par la suite, nous avons commencé à modifier chaque attribut en fonction des actions du joueur.

Action	Fatigue	Patience	Agressivité	Munitions	Pièges
Attaque	+	-	+		
Tir	+	-	+	-	
Placer Pieu	-	+	-		-
Recharge	-	+	-	+	
Ravitaillement	-	+	-	+	
Poursuite cible	+	-	+		
Poursuite Position	+	-	+		
Patrouille	-	+	-		

Tout semble idéal, mais malheureusement nous n'avons pas appliqué les bonnes méthodes pour implanter l'utilité. Effectivement, nous avons essayé d'implanter tous les attributs et formules d'utilité en même temps. Cela a provoqué un effet de chaîne.



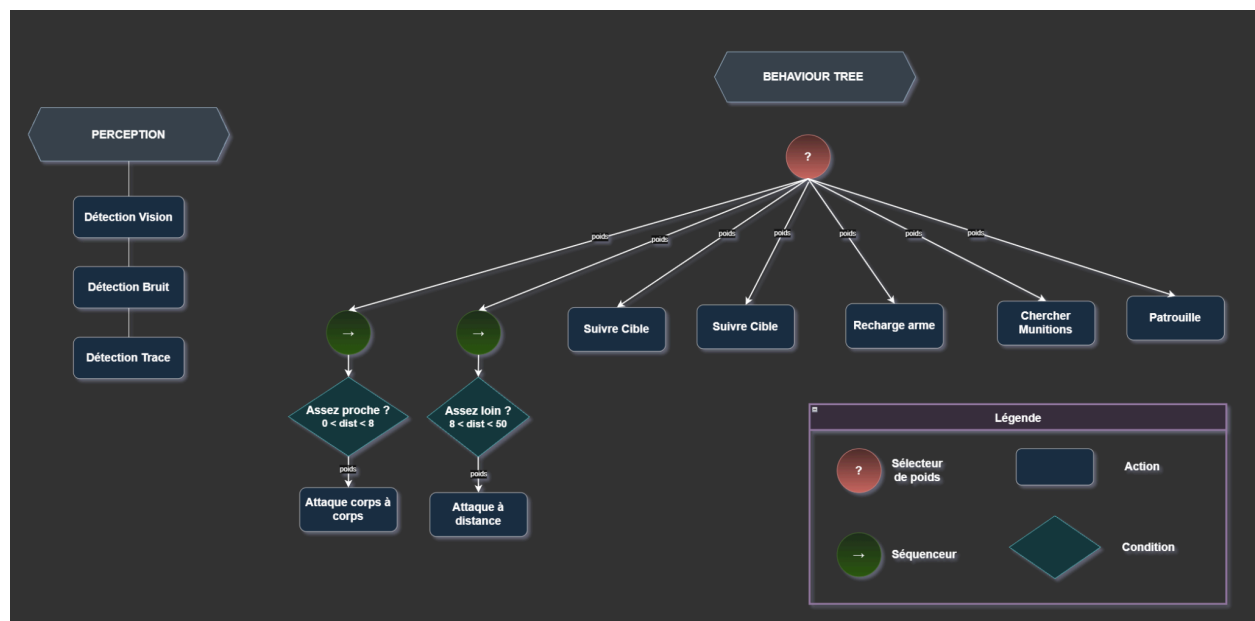
Pour comprendre le problème, prenons un exemple, l'action **Tir**. Cette action aurait comme formule $Utility(Tir) = Agressivité - DistanceJoueur$.

Eh bien, d'après notre tableau, l'action **Tir** augmente l'agressivité, or l'agressivité fait partie de notre formule d'utilité. Donc si la modification de l'agressivité est mal réglée, cela pourrait augmenter grandement l'utilité de l'action **Tir** et donc notre chasseur ne ferait que tirer ou inversement.

Nous avons donc mal étudié le problème et avons perdu beaucoup de temps à essayer de calibrer la modification des attributs. A la fin nous avons décidé de bloquer à des valeurs fixes certains attributs comme la fatigue, l'agressivité et la patience. Cette décision nous a permis d'obtenir une version du jeu jouable.

Pendant ce temps les pièges ont été faits.

Pour conclure nous avons voulu faire gros dès le départ par peur de ne pas faire assez et nous nous sommes retrouvés à l'inverse bloqués dans une complexité bien trop élevée et avons perdu beaucoup de temps à la régler.



9. Schéma du BT avec poids

7. Répartition du travail

Hugo	Noah	Emma
Structure arbre de comportement	Animation du chasseur	Système de vision du chasseur
Multijoueur / Système lobby	Début du système de buissons	Attaque à distance
Synchronisation données client/serveur	Interface utilisateur	Gestion des munitions
Gestion de mort du lapin	Actions du joueur	Ravitaillement des munitions
Apparition des carottes	Début de la map	Système de pièges
Sons du lapin et des objets		Intégration dans le BT
Modèle chasseur		Pathfinding
Mouvement lapin / Caméra		
Utility		

8. Planning de déroulement du projet durant les 3 itérations de ce semestre

Itération 1	Itération 2	Itération 3
Mise en place des bases du projet	Amélioration du Behavior tree	Début de l'Utility AI
Création du lapin jouable	Détection, poursuite et combat	Finalisation des pièges
Première version du chasseur	Gestion des munitions	Implémentation du terrier
Carte de test	Début des pièges	Amélioration des animations
Prise en main de Lua et de Roblox	Multijoueur	Tests et équilibrage

9. Élément original

Noah Laghlali

L'élément dont je suis le plus fier dans The Silent Hunt est la gestion de la barre de faim. J'ai dû apprendre à gérer les relations entre client et serveur, et ce dès le début du projet.

Le fonctionnement est le suivant : pour éviter une surexploitation inutile du serveur, et un envoi de paquets très élevé, la barre de faim est gérée par le client, comme ça, le serveur est soulagé de beaucoup de requêtes.

Et pour être sûr qu'il n'y ait pas de problème de synchronisation entre les joueurs, le serveur envoie tout de même un paquet toutes les 10 secondes, mais également quand une carotte est mangée. Ainsi, les joueurs ne peuvent pas modifier leurs clients pour, par exemple, arrêter de faire baisser leur barre de faim.

J'ai également créé toute la partie graphique de cette barre, en ajoutant le logo de carotte, et en mettant en place l'affichage dynamique, avec la barre qui diminue avec le temps, et qui augmente quand une carotte est mangée.

J'ai aussi totalement fait la carotte qui permet d'augmenter la barre de faim. J'ai mis en place le modèle 3D, ajouté le système d'interaction que j'ai dû apprendre, car il est propre à Roblox Studio, et fait en sorte que la valeur de faim redonnée soit facilement changeable, pour plus tard ajouter des carottes de différente taille qui nourrissent plus ou moins.

Voir 4. *Diagramme de séquence du système de satiété* dans la présentation de l'itération 1.

Emma Klingler

L'élément dont je suis la plus fière dans le projet *The Silent Hunt* est le système de combat du chasseur. Mon objectif n'était pas simplement d'ajouter une attaque à distance, mais de créer un comportement cohérent et crédible.

Pour la poursuite du joueur, j'ai utilisé le PathfindingService afin que le chasseur puisse contourner les obstacles de manière naturelle. Cela évite un déplacement trop simple ou irréaliste et rend la traque plus immersive.

Concernant le tir à distance, j'ai mis en place un système de raycast pour vérifier la ligne de vue avant chaque tir. Ainsi, le chasseur ne peut pas tirer à travers les arbres ou les objets. Ce choix permet d'éviter des comportements incohérents et renforce la crédibilité de l'intelligence artificielle.

J'ai également développé une gestion complète des munitions, avec un chargeur, une réserve, un système de rechargement et un ravitaillement. Le chasseur doit donc adapter ses actions en fonction de ses ressources, ce qui ajoute une dimension stratégique à son comportement.

Enfin, tout ce système est intégré dans le Behaviour Tree. Les actions ne bloquent pas l'exécution et retournent un statut (RUNNING, SUCCESS, FAILURE), ce qui permet un fonctionnement fluide et structuré.

Je suis fière de cet élément car il ne s'agit pas d'une fonctionnalité isolée, mais d'un ensemble cohérent qui combine perception, déplacement, gestion de ressources et logique décisionnelle. Cela montre l'évolution du projet vers une IA plus aboutie et plus réaliste.

Hugo Rochedreux

Mon élément original est le Behaviour Tree. Ce qui me rend fier n'est pas simplement d'avoir réalisé un BT, mais d'avoir conçu un BT propre et structuré de manière professionnelle :

- Aucune logique métier n'est présente dans les nœuds. Toute logique, tout calcul et toute action sont réalisés dans le chasseur, ce qui permet de réutiliser le BT pour n'importe quelle entité. Il suffit simplement d'adapter la hiérarchie et les fonctions dans la nouvelle entité.
- Aucun blocage. Les BT classiques sont souvent bloquants, avec un wait() pour attendre la fin d'une action ou autre. Le nôtre continue de boucler à chaque frame, donc rien n'est bloquant. Chaque action poursuit son exécution en retournant RUNNING jusqu'à atteindre son objectif.
- Une excellente modularité. Pour ajouter une action au chasseur, c'est extrêmement simple : il suffit d'ajouter un nœud dans l'arbre et une fonction dans le chasseur.

De même, pour modifier la priorité des actions, il suffit d'organiser la hiérarchie de l'arbre. Avec l'ajout des poids à l'itération 3, notre BT devient réellement plus complet et plus complexe. Cela m'impressionne, car à partir d'un concept simple au départ, nous avons pu développer une IA capable de prendre des décisions intelligentes et adaptées au contexte.

Je suis également fier de mon chasseur, qui est un modèle 3D réalisé à la main, avec des animations que j'ai également créées moi-même.



10. Image du chasseur

10. Objectifs à atteindre pour la fin du projet

Les prochaines étapes du projet se concentrent sur l'IA, l'expérience de jeu et la finition globale.

10.1 IA du chasseur (priorité n°1)

L'objectif est de rendre le chasseur plus intelligent, cohérent et adaptatif.

- L'ajout d'une mémoire comportementale (zones suspectes, habitudes du joueur)
- Un apprentissage progressif : seuil de bruit adaptable, patrouilles dynamiques, placement intelligent de pièges
- Une anticipation simple des déplacements du lapin
- Un ajustement automatique de l'agressivité selon la partie

10.2 IA du lapin

Les lapins non joueurs doivent passer d'un comportement aléatoire à un comportement logique.

- Eviter le chasseur
- Chercher de la nourriture
- Utiliser les terriers
- Fuir en cas de stress élevé

L'objectif est de rendre le jeu plus réaliste et de fournir un meilleur entraînement à l'IA du chasseur.

10.3 HUD et visuels

Plusieurs éléments visuels doivent être finalisés :

- jauges (vie, faim...)
- système jour/nuit
- alertes visuelles (poursuite, piège, événements importants)

10.4 Missions secondaires

Quelques missions seront intégrées, par exemple :

- manger une carotte rare
- atteindre un terrier éloigné
- sauver un lapin blessé

Des récompenses seront ajoutées (réduction de stress, vision du chasseur, bonus de faim/vie).

10.5 Multijoueur

Le mode multijoueur doit être amélioré :

- Ajout d'éléments coopératifs (aider un autre lapin, chat vocal...)
- Ajout d'éléments non coopératifs (bloquer l'accès d'un terrier par exemple)

Le joueur aura alors le choix entre être un bon lapin et sauver ses amis ou alors tout faire pour survivre, qu'à les sacrifier.

10.6 Carte finale

La carte de test sera remplacée par une carte complète comprenant :

- forêt, clairière, rivière, rochers...
- placement réfléchi des ressources, terriers et zones du chasseur
- terriers finalisés à 100% (téléportation, cachette...)

10.7 Finition et optimisation

La dernière phase portera sur :

- l'équilibrage des systèmes (faim, stress, bruit)
- l'optimisation des scripts client/serveur
- le nettoyage du code
- la documentation et le rapport final

11. Planning pour les trois itérations restantes

Les trois premières itérations étant terminées, il reste à planifier les trois dernières :

11.1 Itération 4 : Ajustement des poids

Le but de cette itération est simple : trouver un optimum pour une action simple du chasseur. Effectivement, nous avons remarqué lors de l'itération 3 que l'implémentation de l'Utility n'est pas simple. Le but de cette itération 4 serait alors de se concentrer seulement sur quelques attributs :

- Balles chargeur
- Fatigue
- Distance du lapin

Ainsi que de se concentrer seulement sur deux actions :

- Tirer
- Recharger

Cela va nous permettre de trouver un optimum plus facilement pour ces valeurs et donc de pouvoir ensuite travailler avec plus d'attributs et sur plus d'actions.

Les autres tâches en parallèle sont :

- terminer le système de terriers
- commencer l'IA du lapin (si le temps)

11.2 Itération 5 : IA adaptative et apprentissage léger

Cette phase se concentre sur l'intelligence artificielle avancée.

Les travaux prévus incluent :

- implémentation de la mémoire comportementale du chasseur
- adaptation dynamique des patrouilles
- tests d'apprentissage
- implémenter l'IA du lapin

Objectif de fin d'itération : une version avec une IA plus dynamique, capable de simulations IA contre IA

11.3 Itération 6 : Amélioration finale et préparation de la soutenance

La dernière itération vise à finaliser l'expérience globale et préparer la présentation.

Les tâches principales sont :

- finalisation du HUD
- ajout d'effets visuels et sonores
- équilibrage des systèmes (faim, stress, bruit)
- équilibrage des calculs de poids
- finalisation du rapport, de la documentation et de la soutenance

Objectif de fin d'itération : un jeu complet, stable et prêt pour la soutenance.

Conclusion

Ce semestre a permis de poser des bases solides pour *The Silent Hunt*. Les trois itérations ont progressivement construit le gameplay, l'architecture réseau et l'intelligence artificielle du chasseur. Malgré les difficultés techniques et organisationnelles, les objectifs essentiels ont été atteints : un prototype jouable, un Behaviour Tree complet et un système multijoueur fonctionnel.

L'itération 3 a mis en évidence la complexité de l'Utility AI et la nécessité d'une approche plus progressive. Les pièges et les améliorations graphiques ont néanmoins enrichi le jeu et renforcé la cohérence du projet. L'ensemble des choix réalisés reste conforme à l'étude préalable, qui prévoyait une IA basée sur un Behaviour Tree complété par un système d'Utility, ainsi qu'un gameplay centré sur la traque, la survie et l'utilisation de l'environnement.

Le travail réalisé constitue une base stable pour la suite du développement. Les prochaines itérations se concentreront sur l'ajustement des poids, l'amélioration de l'IA du lapin, l'optimisation générale, l'ajout d'éléments comme le terrier et la préparation de la version finale.